

Simon Says Memory Game

RTL Design, Implementation, and Verification Report

Prepared for: **Module 4 — Digital Design**

Name: **Muhammad Kaleem Faryad**

Mentor: **Eman Nasar**

Date: 28/07/2026

Table of Contents

1. Introduction.....	4
2. Project Overview	5
3. RTL Module Implementation	6
3.1 Clock Divider.....	6
3.2 Linear Feedback Shift Register (LFSR).....	6
3.3 Sequence Memory	6
3.4 Display Index Register.....	7
3.5 Input Index Register.....	7
3.6 Sequence Length Register.....	8
3.7 Score Register	8
3.8 Button Pulse Generator	8
3.9 Button Encoder	8
3.10 Comparator	9
3.11 LED Decoder	9
3.12 Seven Segment Decoder	9
3.13 Simon FSM	9
.....	10
3.14 Top-Level Module (simon_says).....	10
4. Functional Verification and Testbench Design.....	12
4.1 Verification Strategy	12
4.2 Testbench Architecture	12
4.3 Clock Generation	13
4.4 Waveform Dumping	13
4.5 Simulation Monitor.....	13
4.6 Button Press Task	13
4.7 Test Sequence	14
Step 1 – Reset Verification	14
Step 2 – Game Start Verification	14
Step 3 – Pattern Display Verification	14
Step 4 – Correct Button Verification	14
Step 5 – Score Verification	14
Step 6 – Next Level Verification	15

Step 7 – Game Over Verification.....	15
4.8 Self-Checking Verification	15
4.9 Simulation Results	15
5. Verification and Testing Methodology	17
5.1 Verification Strategy	17
5.2 Unit Testing	17
5.3 Top-Level Functional Verification	17
5.4 Self-Checking Testbench	18
5.5 Debugging Improvements.....	18
5.6 Waveform Verification	19
5.7 Final Verification Result.....	19
6. Results and Discussion	20
7. Conclusion and Future Work	21
7.1 Conclusion	21
7.2 Future Work.....	21

1. Introduction

The Simon Says Memory Game is a digital memory challenge in which a player observes a sequence of illuminated LEDs and attempts to reproduce the same sequence using push buttons. After each successful round, the system appends one additional random element to the stored sequence, increasing the game's difficulty. The game continues until the player presses an incorrect button, at which point the game enters the Game Over state.

The objective of this project is to design and implement the Simon Says Memory Game using SystemVerilog by applying digital design concepts introduced in Module 4. The implementation follows a modular hardware design methodology in which the controller and datapath are implemented as separate modules. A Moore Finite State Machine (FSM) controls the overall game flow, while dedicated hardware modules perform sequence generation, storage, comparison, score keeping, and display operations.

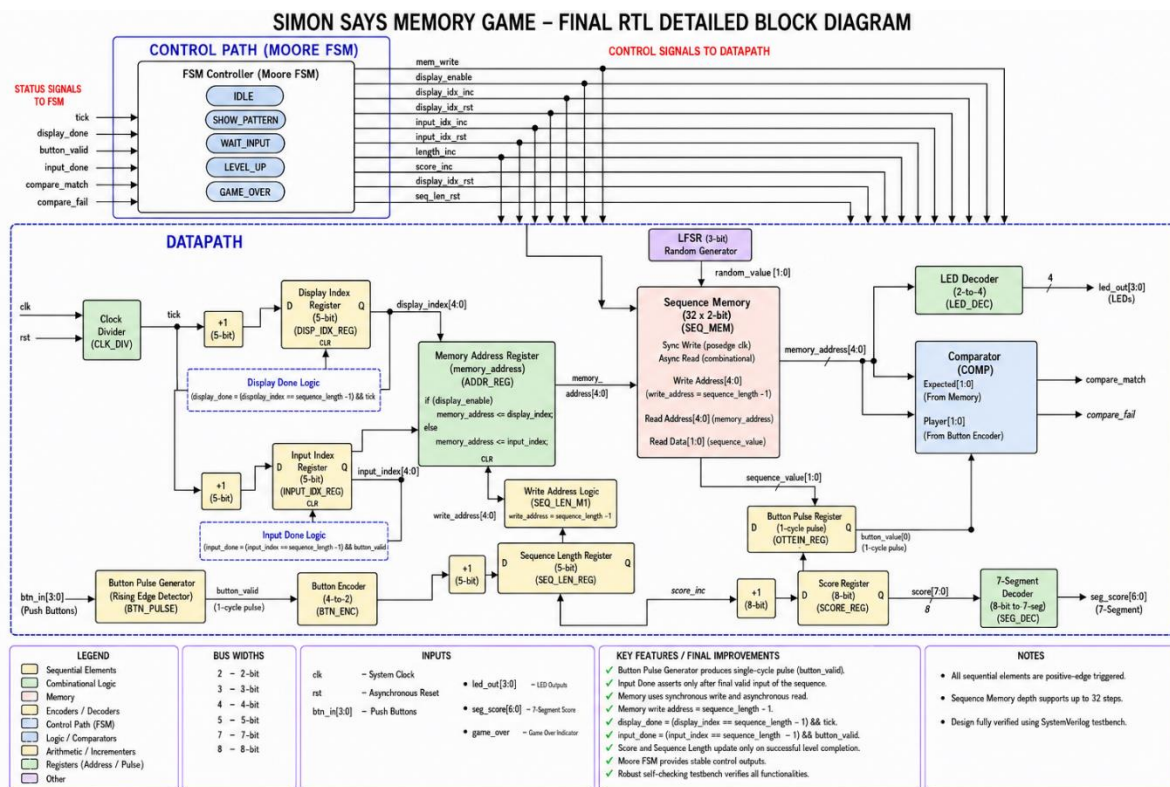
The project was implemented and verified using EDA Playground with the Synopsys VCS simulator. Functional verification was performed through a self-checking SystemVerilog testbench, and signal behavior was analyzed using EPWave. During implementation, several debugging improvements were incorporated, including edge detection for button presses, improved memory addressing, synchronization of datapath operations, and automatic PASS/FAIL verification in the testbench.

2. Project Overview

The Simon Says Memory Game is implemented as a modular Register Transfer Level (RTL) digital system using SystemVerilog. The design follows a controller-datapath architecture in which a Moore Finite State Machine (FSM) controls the overall game flow while dedicated hardware modules perform specific operations such as random sequence generation, memory storage, input processing, score management, and output display.

The game begins when the player presses the **Start** signal. A pseudo-random value is generated using a Linear Feedback Shift Register (LFSR) and stored in the sequence memory. The controller then displays the stored sequence through the LEDs. The player reproduces the sequence using push buttons, and each button press is compared with the stored sequence. If every input matches correctly, the score and sequence length are incremented, a new random value is appended to the sequence, and the next round begins. If an incorrect button is pressed, the controller transitions to the **GAME_OVER** state and terminates the game.

The design is completely synchronous with respect to the system clock. All sequential elements, including registers, counters, memory write operations, and the FSM state register, are updated on the positive edge of the clock. This approach provides predictable timing behavior and simplifies hardware implementation.



3. RTL Module Implementation

3.1 Clock Divider

The clock divider generates a slower timing pulse from the system clock. The generated **tick** signal controls the speed at which the Simon Says pattern is displayed on the LEDs.

Instead of updating the display every clock cycle, the divider counts clock pulses until a predefined value (DIVIDE_COUNT) is reached. At that instant, it produces a one-clock-cycle pulse (tick) and resets the counter.

Inputs: clk, rst

Output: tick

Design Features:

- Synchronous counter
- Parameterized divide value
- Single-cycle tick generation

EDA Playground link: <https://edaplayground.com/x/fXcV>

3.2 Linear Feedback Shift Register (LFSR)

The LFSR is responsible for generating the pseudo-random sequence used in the game.

A 3-bit shift register with XOR feedback is used. Every time the controller asserts mem_write, the register shifts and produces a new 2-bit random value.

Inputs: clk, rst, enable

Output: random_value

Design Features:

- 3-bit feedback register
- XOR feedback polynomial
- Enabled only when a new sequence element is required

EDA Playground link: https://edaplayground.com/x/X_KC

3.3 Sequence Memory

The sequence memory stores the Simon Says pattern.

Each generated random value is written into memory during the LEVEL_UP state. During gameplay, the memory is read either for displaying the pattern or comparing player input.

The memory contains **32 locations**, each storing a **2-bit value**.

Inputs: Clock, Write Enable, Write Address, Write Data, Read Address

Output: Read Data

Design Decisions:

- Synchronous write
- Asynchronous read
- Separate read and write addresses

Using asynchronous read allows the expected value to become immediately available whenever the read address changes, which simplifies the comparison logic during gameplay.

EDA Playground link: <https://edaplayground.com/x/hntv>

3.4 Display Index Register

The display index register keeps track of the sequence element currently being shown on the LEDs.

Whenever the FSM asserts `display_idx_inc`, the index increments. When a new level begins, the register is cleared to zero.

Inputs: Clock, Reset, Increment, Clear

Output: Display Index

Design Features:

- 5-bit register
- Synchronous increment
- Synchronous clear

EDA Playground link: <https://edaplayground.com/x/cwtS>

3.5 Input Index Register

The input index register records which sequence element the player is currently entering. The index advances only after:

- a valid button press, and
- a successful comparison.

This prevents incorrect advancement when the player holds a button.

Inputs: Clock, Reset, Increment, Clear

Output: Input Index

EDA Playground link: https://edaplayground.com/x/qc_i

3.6 Sequence Length Register

This register stores the number of sequence elements currently used by the game. The initial value is **1**. After every successfully completed level, the controller increments the sequence length by one.

Inputs: Clock, Reset, Length Increment

Output: Sequence Length

EDA Playground link: <https://edaplayground.com/x/Lab6>

3.7 Score Register

The score register stores the player's score. Whenever the LEVEL_UP state is reached, the score increments by one.

Inputs: Clock, Reset, Score Increment

Output: Score

EDA Playground link: <https://edaplayground.com/x/AKnk>

3.8 Button Pulse Generator

Mechanical push buttons remain high for multiple clock cycles when pressed. If the raw button signal were used directly, the input index could increment several times for a single press.

To solve this problem, the design includes a **Button Pulse Generator**, which compares the current button input with its previous value. A one-clock-cycle pulse (button_valid) is generated only on the rising edge of a button press.

Inputs: Clock, Reset, Button Inputs

Output: Button Valid Pulse

EDA Playground link: <https://edaplayground.com/x/Ja8X>

Design Improvement: This module was introduced during debugging to eliminate repeated button detections caused by long button presses.

3.9 Button Encoder

The button encoder converts the one-hot button inputs into a compact 2-bit binary value.

Button	Encoded Value
0001	00
0010	01
0100	10
1000	11

This encoded value is sent to the comparator.

EDA Playground link: <https://edaplayground.com/x/pXSx>

3.10 Comparator

The comparator checks whether the player's button matches the expected sequence value stored in memory. It generates two status signals:

- compare_match
- compare_fail

These signals are used by the FSM to determine whether the player continues the game or transitions to the GAME_OVER state.

EDA Playground link: <https://edaplayground.com/x/Xssk>

3.11 LED Decoder

The LED decoder converts the 2-bit sequence value into a one-hot LED output. Each stored value activates exactly one LED.

Value	LED Output
00	0001
01	0010
10	0100
11	1000

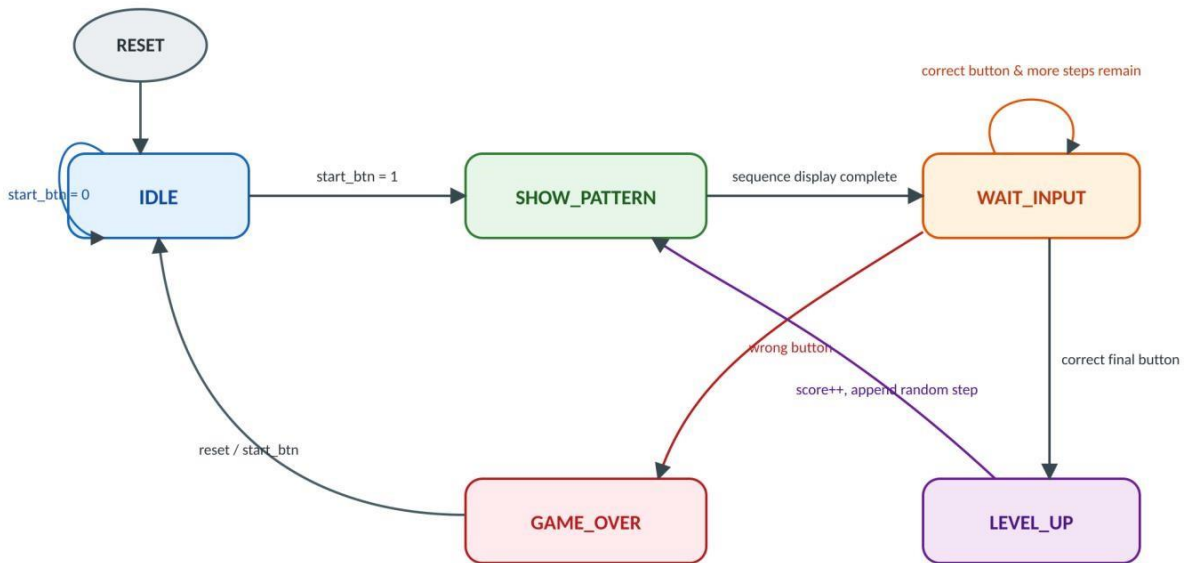
EDA Playground link: <https://edaplayground.com/x/LjWs>

3.12 Seven Segment Decoder

The seven-segment decoder converts the current score into the corresponding seven-segment display pattern. Scores from **0 to 9** are supported.

EDA Playground link: <https://edaplayground.com/x/Asa9>

3.13 Simon FSM



The Simon FSM acts as the controller for the complete game. It implements five states:

- IDLE
- SHOW_PATTERN
- WAIT_INPUT
- LEVEL_UP
- GAME_OVER

The FSM generates all control signals required by the datapath, including:

- Display enable
- Display index increment
- Input index increment
- Memory write enable
- Score increment
- Sequence length increment
- Game over signal

The FSM follows the Moore machine model, where outputs depend only on the current state.

During implementation, the WAIT_INPUT state was improved by checking button_valid before evaluating comparison results. This prevented false GAME_OVER transitions when no button was pressed.

EDA Playground link: <https://edaplayground.com/x/gLhZ>

3.14 Top-Level Module (simon_says)

The `simon_says` module integrates all RTL components into a complete digital system. Its responsibilities include:

- Connecting the controller and datapath.
- Selecting the correct memory address during display and input phases.
- Coordinating random sequence generation.
- Comparing player input with stored data.
- Updating score and sequence length.
- Driving LEDs and seven-segment display.
- Detecting game completion.

The top-level module serves as the complete hardware implementation of the Simon Says Memory Game.

EDA Playground link: <https://edaplayground.com/x/bKrQ>

4. Functional Verification and Testbench Design

4.1 Verification Strategy

After completing the RTL implementation, a self-checking SystemVerilog testbench was developed to verify the complete functionality of the Simon Says Memory Game. The objective of the testbench was to ensure that every major function of the design behaved correctly without requiring manual waveform inspection.

The verification focused on the following features:

- Correct reset behavior
- Successful game start
- Pattern display
- Correct button detection
- Sequence comparison
- Score update
- Sequence length update
- Level transition
- Game over detection

The testbench automatically reports **PASS** or **FAIL** messages for each verification step.

4.2 Testbench Architecture

The testbench instantiates the complete `simon_says` top-level module and generates all required stimulus signals. The following signals are driven by the testbench:

Signal	Purpose
clk	System clock
rst	Initializes the entire design
start	Starts a new game
btn_in	Simulates player button presses

The following outputs are observed:

Signal	Purpose
led_out	Displays Simon sequence
seg	Shows current score
game_over	Indicates end of game

The testbench acts as a virtual player by automatically pressing buttons based on the stored Simon sequence.

4.3 Clock Generation

A free-running clock is generated using an always block.

```
always #5 clk = ~clk;
```

This produces a **10 ns clock period**, corresponding to a **100 MHz** simulation clock. All sequential modules in the design operate using this clock.

4.4 Waveform Dumping

Simulation waveforms are generated using the following SystemVerilog system tasks:

```
$dumpfile("dump.vcd");  
$dumpvars(0, tb_simon_says);
```

These commands create a VCD file that can be viewed using EPWave to inspect internal signals during simulation.

4.5 Simulation Monitor

The testbench continuously prints important internal signals throughout simulation using the \$monitor task. The monitor displays:

- Current FSM state
- Current score
- Sequence length
- Display index
- Input index
- LED outputs
- Game over status

This provides a real-time textual view of system operation alongside the waveform viewer.

4.6 Button Press Task

To simplify stimulus generation, a reusable task named `press_button` was created. The task performs the following operations:

1. Drives one button high.
2. Waits one clock cycle.
3. Releases the button.
4. Waits one additional clock cycle.

Using a task improves readability and avoids repeated code throughout the testbench.

4.7 Test Sequence

The verification process follows the complete game flow.

Step 1 – Reset Verification

The design is held in reset for four clock cycles. After reset is released, all internal registers should return to their initial values. Expected behavior:

- FSM enters IDLE state.
- Score becomes 0.
- Sequence length becomes 1.
- Display and input indices become 0.

Step 2 – Game Start Verification

The start signal is asserted for one clock cycle. Expected behavior:

- FSM transitions from IDLE to SHOW_PATTERN.
- First random sequence value is written into memory.

Step 3 – Pattern Display Verification

The testbench waits until the FSM enters the WAIT_INPUT state. Expected behavior:

- LEDs display the generated Simon pattern.
- Display index increments correctly.
- Display phase completes successfully.

Step 4 – Correct Button Verification

The testbench reads the expected value stored in sequence memory. Based on that value, the correct button is automatically pressed. Expected behavior:

- Comparator reports a successful match.
- Input index increments.
- FSM transitions to LEVEL_UP.

Step 5 – Score Verification

Once the FSM reaches LEVEL_UP, the testbench waits one additional clock cycle before checking sequential registers. The following conditions are verified automatically:

- Score increments from 0 to 1.
- Sequence length increments from 1 to 2.

PASS messages are displayed if both conditions are satisfied.

Step 6 – Next Level Verification

The FSM automatically returns to the SHOW_PATTERN state. The updated sequence is displayed again with the newly added random value. The testbench waits until the FSM enters WAIT_INPUT for the second level.

Step 7 – Game Over Verification

An intentionally incorrect button is pressed. Expected behavior:

- Comparator detects a mismatch.
- FSM transitions to GAME_OVER.
- Game over signal becomes active.

The testbench reports a PASS message once the game over condition is detected.

4.8 Self-Checking Verification

Unlike manual testing, the developed testbench automatically verifies the design using conditional statements. Examples include checking:

- Score increment
- Sequence length increment
- Successful game over detection

If any condition fails, the simulation immediately terminates with an appropriate FAIL message. This approach greatly improves verification efficiency and eliminates manual result checking.

4.9 Simulation Results

Simulation was performed using **Synopsys VCS** on **EDA Playground**. The generated console output confirmed that:

- Reset completed successfully.
- Game started correctly.
- Simon pattern was displayed.
- Correct button press advanced the level.
- Score increased successfully.
- Sequence length increased correctly.
- Incorrect button press generated GAME_OVER.

- All self-checks passed successfully.

The accompanying EPWave waveform also verified correct timing relationships between the FSM, datapath, memory, comparator, and control signals. Therefore, the RTL implementation successfully satisfies all required functional specifications of the Simon Says Memory Game.

5. Verification and Testing Methodology

5.1 Verification Strategy

Building on the testbench design and simulation results presented in Sections 5 and 6, verification was carried out using a hierarchical strategy: each RTL module was first tested independently (unit testing), after which the modules were integrated into the complete `simon_says` design and verified using the top-level self-checking testbench. This hierarchical approach simplified debugging, since errors could be isolated to a specific module before system integration.

5.2 Unit Testing

Each RTL module was tested separately before system integration. The following modules were verified individually:

Module	Purpose	Verification
Clock Divider	Generates slower tick pulses	Tick generated after correct number of clock cycles
LFSR	Generates pseudo-random sequence	New random value produced after every enable
Sequence Memory	Stores Simon sequence	Correct read/write operations verified
Display Index Register	Controls display position	Increment and reset operations verified
Input Index Register	Tracks player's input	Correct increment/reset verified
Sequence Length Register	Stores current level	Increment verified after successful round
Score Register	Stores player score	Score increment verified
LED Decoder	Converts 2-bit value to LEDs	Correct LED activated
Button Encoder	Converts push buttons into binary value	Correct encoding verified
Comparator	Compares expected and player input	Match and fail conditions verified
Seven Segment Decoder	Displays score	Correct digit displayed
Button Pulse Generator	Detects new button presses	Single pulse generated per press
FSM	Controls complete game flow	All state transitions verified

Testing each module independently simplified debugging and ensured that errors were isolated before system integration.

5.3 Top-Level Functional Verification

After integrating all modules into the `simon_says` top module, the complete Simon Says game was verified following the actual game behavior described in Section 5.7. The table below summarizes the top-level test cases; the corresponding waveform evidence for each is presented in Section 6.

Test Case	Expected Result	Status
System Reset	State = IDLE; Score = 0; Sequence Length = 1; Display/Input Index = 0	PASS
Start Game	FSM transitions to SHOW_PATTERN; random value written to memory; LEDs display first pattern	PASS
Correct Button Press	Comparator reports Match; Input Index increments; FSM enters LEVEL_UP	PASS
Level Progression	Score and Sequence Length increment; new random value added; indices reset	PASS
Incorrect Button Press	Comparator reports Fail; FSM enters GAME_OVER; Game Over output asserted	PASS

5.4 Self-Checking Testbench

Instead of relying only on waveform observation, the self-checking testbench automatically verified key system events, including correct score increment, correct sequence length increment, correct transition to GAME_OVER, and correct FSM state transitions. PASS and FAIL messages were displayed automatically during simulation, for example:

```
PASS : SCORE UPDATED
PASS : SEQUENCE LENGTH UPDATED
PASS : GAME OVER DETECTED
PASS : SIMON SAYS GAME VERIFIED
```

This automated verification significantly reduced manual debugging effort and increased confidence in the correctness of the design.

5.5 Debugging Improvements

Several improvements were made during verification to eliminate functional issues discovered during simulation. The following modifications were incorporated into the final design:

- Added the `button_pulse` module to generate a single-cycle pulse for each button press, preventing multiple increments while a button remained pressed.
- Modified `input_done` so that it asserts only after the final valid player input.
- Corrected the memory write address width using the following assignment:

```
.write_address(sequence_length - 5'd1)
```

- Changed the sequence memory to use asynchronous read for immediate availability of stored values.
- Corrected testbench timing so that register values were checked after the appropriate clock edge.
- Simplified monitor outputs for easier debugging.
- Added automatic PASS/FAIL messages for key verification points.

These improvements resolved timing-related issues and ensured reliable system behavior.

5.6 Waveform Verification

Waveform analysis was performed using EPWave to verify timing relationships between different modules. The following signals were monitored:

- Clock, Reset, Start
- FSM Current State
- Display Index, Input Index
- Sequence Length, Score
- Memory Output, LED Output
- Game Over

Waveform inspection confirmed correct state transitions, proper register updates, correct memory read/write operations, proper synchronization between modules, and successful progression through multiple game levels.

5.7 Final Verification Result

The completed Simon Says RTL design successfully passed all functional verification tests. The final simulation demonstrated:

- Successful game initialization
- Correct sequence generation
- Proper LED display
- Accurate player input comparison
- Correct score updates
- Proper level advancement
- Reliable game-over detection

No functional errors remained after debugging, confirming that the RTL implementation satisfied the project requirements.

6. Results and Discussion

The Simon Says Memory Game was successfully implemented and verified using SystemVerilog. All RTL modules were integrated into a single top-level design and simulated using Synopsys VCS on EDA Playground.

The simulation results confirmed that the system performs all required functions correctly. During execution, the FSM transitioned through the expected states (IDLE, SHOW_PATTERN, WAIT_INPUT, LEVEL_UP, and GAME_OVER), while the datapath modules correctly generated, stored, displayed, and compared the Simon sequence.

The self-checking testbench verified the following functionalities:

- Successful system reset
- Correct game initialization
- Pattern generation and display
- Correct player input detection
- Score increment after a successful round
- Sequence length increment for the next level
- Game over detection on incorrect input

During development, several issues were identified and resolved. A **button pulse generator** was added to prevent multiple detections from a single button press. The input completion logic was modified so that the FSM advances only after the final valid button press. The **memory write address width** was corrected, and the testbench timing was improved by checking sequential outputs after the appropriate clock edge. These modifications resulted in stable and reliable simulation behavior.

The final simulation completed successfully without functional errors, and all self-checking test cases reported PASS messages. The generated EPWave waveforms also confirmed correct synchronization between the control unit and datapath throughout the game execution.

Overall, the implementation satisfies the functional requirements of the Simon Says Memory Game and demonstrates correct operation of the complete RTL design.

7. Conclusion and Future Work

7.1 Conclusion

This project presented the design and implementation of a Simon Says Memory Game using SystemVerilog. The system was developed using a modular RTL design approach consisting of a finite state machine, datapath components, sequence memory, random sequence generator, input processing, and display modules.

Each module was verified individually before integrating the complete system. A self-checking SystemVerilog testbench was used to validate the overall functionality, including game initialization, pattern generation, user input verification, score updating, level progression, and game-over detection. Simulation results obtained using Synopsys VCS and EPWave confirmed that the design operates according to the specified requirements.

The project demonstrates the practical application of finite state machines, sequential logic, combinational logic, and verification techniques in digital system design.

7.2 Future Work

The current implementation can be further enhanced in several ways:

- Support longer and more challenging game sequences.
- Add multiple difficulty levels by varying the display speed.
- Display the score on multiple seven-segment displays.
- Implement sound effects for button presses and game events.
- Interface the design with an FPGA development board using physical LEDs, push buttons, and seven-segment displays.
- Add a restart option after the Game Over state without requiring a system reset.

These enhancements would improve the functionality and user experience while making the project suitable for real hardware implementation.